

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..24

CUSTOMIZATIONS

 Copy as Markdown

Customize GitHub sign-in

This page covers additional configuration options for the GitHub identity provider beyond the basic setup. On the server, you can control how credentials are loaded and hook into the sign-in flow with callbacks. In your app, you can configure client IDs and redirect URIs, and customize the sign-in UI. Use the `GitHubSignInWidget` to display the GitHub sign-in flow in your own custom UI, or the `GitHubAuthController` to build a completely custom authentication interface.

Server configuration

The options in this section are set on the server when you register the GitHub identity provider.

Configuration options

Below is a non-exhaustive list of some of the most common configuration options. For more details on all options, check the `GitHubIdpConfig` in-code documentation.

The GitHub identity provider can be configured using one of two classes:

- `GitHubIdpConfigFromPasswords`: Automatically loads the client ID and secret from the `githubClientId` and `githubClientSecret` keys in `passwords.yaml` (or the matching `SERVERPOD_PASSWORD_*` environment variables). This is the class used in the [setup guide](#) and is recommended for most projects.
- `GitHubIdpConfig`: Requires you to pass the client ID and secret directly. Use this when you load credentials from a custom source, such as a secrets manager or a programmatically constructed config.

The `GitHubIdpConfigFromPasswords` class is a convenience wrapper around `GitHubIdpConfig` that handles credential loading for you.

Both classes accept the same optional callbacks shown in the sections below. The examples on this page use `GitHubIdpConfigFromPasswords` unless the section specifically demonstrates manual credential



Load credentials using GitHubIdpConfig

When using `GitHubIdpConfig`, you must provide the client ID and secret explicitly. Read them from any source you want:

```
final githubIdpConfig = GitHubIdpConfig(
  clientId: pod.getPassword('githubClientId')!,
  clientSecret: pod.getPassword('githubClientSecret')!,
);
```

Or from a secrets manager, hard-coded values for tests, or a custom loader:

```
final githubIdpConfig = GitHubIdpConfig(
  clientId: await mySecretsManager.fetch('github-client-id'),
  clientSecret: await mySecretsManager.fetch('github-client-secret'),
);
```

Custom account validation

You can customize the validation for GitHub account details before allowing sign-in. By default, the validation only checks that the received account details contain a non-empty `userIdentifier`.

```
final githubIdpConfig = GitHubIdpConfigFromPasswords(
  githubAccountDetailsValidation: (accountDetails) {
    // Throw an exception if account doesn't meet custom requirements
    if (accountDetails.userIdentifier.isEmpty) {
      throw GitHubUserInfoMissingDataException();
    }
  },
);
```

NOTE

GitHub users can keep their email private, so `email` may be `null` even for valid accounts. Similarly, `name` is optional on GitHub profiles. To avoid blocking real users with private profiles from signing in, adjust your validation function with care.

GitHubAccountDetails

The `githubAccountDetailsValidation` callback receives a `GitHubAccountDetails` record with the following properties:

Property	Type	Description
<code>userIdentifier</code>	<code>String</code>	The GitHub user's unique identifier (UID)
<code>email</code>	<code>String?</code>	The user's email address (may be <code>null</code> if private)
<code>name</code>	<code>String?</code>	The user's display name from GitHub
<code>image</code>	<code>Uri?</code>	URL to the user's profile image

Example of accessing these properties:

```
githubAccountDetailsValidation: (accountDetails) {
  print('GitHub UID: ${accountDetails.userIdentifier}');
  print('Email: ${accountDetails.email}');
  print('Display name: ${accountDetails.name}');
  print('Profile image: ${accountDetails.image}');

  // Custom validation logic
  if (accountDetails.email == null) {
    throw GitHubUserInfoMissingDataException();
  }
},
```

Accessing GitHub APIs on the server

On the server side, you can call GitHub's REST API using the access token returned by sign-in. The `getExtraGitHubInfoCallback` on `GitHubIdpConfig` receives the access token on every authentication attempt and can be used to fetch and store additional user data:

```
import 'package:http/http.dart' as http;

final githubIdpConfig = GitHubIdpConfigFromPasswords(
  getExtraGitHubInfoCallback: (session, {
    required accountDetails,
    required accessToken,
    required transaction,
  }) async {
    final response = await http.get(
      Uri.https('api.github.com', '/user/orgs'),
      headers: {
        'Authorization': 'Bearer $accessToken',
        'Accept': 'application/vnd.github+json',
      },
    );
    // Parse response and store organization membership in your own table,
    // linked by accountDetails.userIdentifier.
```

```
},  
);
```

⚠ WARNING

Do not create `GitHubAccount`, `UserProfile`, or `AuthUser` models inside this callback. The authentication flow already creates them. Creating them here breaks new-account detection and skips critical setup steps. Store any extra data in your own custom tables, linked by `accountDetails.userIdentifier`.

ℹ INFO

This callback runs on **every** sign-in, not only the first. Keep operations lightweight or guard expensive work behind a check for whether the data already exists. Guard external calls with `try/catch`, because an uncaught exception in the callback makes the sign-in fail.

Reacting to GitHub account creation

Use the `onAfterGitHubAccountCreated` callback to run logic after a new GitHub account has been created and linked to an auth user. This callback only fires for new accounts, not returning users.

This callback is complementary to the global `onAfterAuthUserCreated` hook and is for side-effects specific to a GitHub sign-in, like storing GitHub-specific analytics or sending a GitHub-themed welcome email.

```
final githubIdpConfig = GitHubIdpConfigFromPasswords(  
  onAfterGitHubAccountCreated: (  
    session,  
    authUser,  
    githubAccount, {  
      required transaction,  
    }) async {  
      // e.g. store additional data, send a welcome email, or log for analytics  
    },  
  ),  
);
```

ℹ INFO

This callback runs inside the same database transaction as the account creation. Throwing an exception inside this callback aborts the process. If you perform external side-effects, guard them with `try/catch` to prevent unwanted failures.

⚠ CAUTION

Scopes you assign here with `AuthServices.instance.authUsers.update()` do not apply to the login that is already in progress, because token issuance uses the scopes loaded before this callback runs. They take effect the next time the user signs in. To force them sooner, revoke the user's tokens so they sign in again. The `onBeforeAuthUserCreated` hook, covered below, assigns scopes at creation time, but it cannot use GitHub data, because `getExtraGitHubInfoCallback` runs after the auth user is created.

Reacting to auth user creation

The `onBeforeAuthUserCreated` and `onAfterAuthUserCreated` hooks are global callbacks configured on `AuthUsersConfig` in `initializeAuthServices`. They are not specific to GitHub. They fire for every identity provider. See [user creation callbacks](#) for full details on both hooks.

GitHubIdpConfig parameter reference

Parameter	Type	Required	Description
<code>clientId</code>	<code>String</code>	Yes	The Client ID from your GitHub App or OAuth App.
<code>clientSecret</code>	<code>String</code>	Yes	The Client Secret generated for your GitHub App or OAuth App.
<code>githubAccountDetailsValidation</code>	<code>GitHubAccountDetailsValidation</code>	No	Custom validation callback for GitHub account details before allowing sign-in. Throws an exception to reject the account. Defaults to validating only that <code>userIdentifier</code> is non-empty.
<code>getExtraGitHubInfoCallback</code>	<code>GetExtraGitHubInfoCallback?</code>	No	Callback that receives the access token after sign-in, allowing you to call additional GitHub APIs and store extra user data. Runs on every sign-in.
<code>onAfterGitHubAccountCreated</code>	<code>AfterGitHubAccountCreatedFunction?</code>	No	Callback invoked after a new GitHub account is created and linked to an auth user. Fires only for new accounts.

App configuration

The options in this section are set in your Flutter app when you initialize the GitHub sign-in service.

Configuring client IDs on the app

Passing client IDs in code

You can pass the `clientId` and `redirectUri` directly when initializing the GitHub sign-in service:

```
await client.auth.initializeGitHubSignIn(  
  clientId: 'your-github-client-id',  
  redirectUri: 'com.example.yourapp://auth',  
);
```

This approach is useful when you need different `redirectUri` values per platform and want to keep them in your Dart code.

Using environment variables

Alternatively, pass them at build time using `--dart-define`. The GitHub sign-in provider supports the following environment variables:

- `GITHUB_CLIENT_ID`: Your GitHub OAuth client ID.
- `GITHUB_REDIRECT_URI`: The callback URI. Use the value matching the platform you build for: a reverse-DNS scheme for mobile, `https://your-domain.com/auth/callback` for Serverpod-hosted Flutter web, or the full `auth.html` URL for separately-hosted Flutter web.

If `clientId` and `redirectUri` are not supplied when initializing the service, the provider automatically falls back to these environment variables.

Example usage:

```
flutter run -d chrome --web-port=49660 \  
  --dart-define="GITHUB_CLIENT_ID=your-github-client-id" \  
  --dart-define="GITHUB_REDIRECT_URI=http://localhost:49660/auth.html"
```

```
flutter build web \  
  --dart-define="GITHUB_CLIENT_ID=your-github-client-id" \  
  --dart-define="GITHUB_REDIRECT_URI=https://my-awesome-  
project.serverpod.space/auth/callback"
```

This approach is useful when you need to:

- Configure different credentials for different build environments (development, staging, production).

- Avoid committing client IDs to version control.
- Inject platform-specific redirect URIs from your CI/CD pipeline.

TIP

You can also set these environment variables in your IDE's run configuration or CI/CD pipeline to avoid passing them manually each time.

Separately-hosted Flutter web

Use this flow when your Flutter web app and Serverpod are on different origins. Common cases: `flutter run -d chrome` locally with Serverpod on a separate port, or a CDN-hosted Flutter build with a separate API server.

1. Place a static `auth.html` file in your Flutter project's `web/` folder. A single copy is shared across every identity provider that uses an OAuth2 redirect, so create it once. Follow [Web callback page \(`auth.html`\)](#) in the authentication setup guide.
2. Run Flutter on a fixed port. The examples use `49660`, but any free port works; keep it consistent everywhere:

```
flutter run -d chrome --web-port=49660
```

3. Register the full `auth.html` URL on your GitHub App's **Callback URL** field (e.g., `http://localhost:49660/auth.html` locally, `https://app.example.com/auth.html` in production). GitHub Apps accept up to 10 callback URLs, so dev and prod entries can coexist with the Serverpod-hosted route and mobile schemes.
4. Pass the same URL to `initializeGitHubSignIn` via the `redirectUri` argument instead of the route URL.

Customize the sign-in button

See [Styling the buttons](#) for how a `buttonStyle` set on `SignInWidget` takes precedence over the appearance arguments shown below.

INFO

The `SignInWidget` uses the `GitHubSignInWidget` internally to display the GitHub sign-in flow. You can also supply a custom `GitHubSignInWidget` to the `SignInWidget` to override the default behavior.

```

SignInWidget(
  client: client,
  githubSignInWidget: GitHubSignInWidget(
    client: client,
    // Shape and label survive inside SignInWidget, unless its buttonStyle
    // sets them. Brand colors do not.
    shape: SignInButtonShape.rounded,
    text: SignInButtonTextVariant.signInWith,
    // A custom widget replaces the built-in handling, so pass your own callbacks.
    onAuthenticated: () { /* ... */ },
    onError: (error) { /* ... */ },
  ),
)

```

Using the `GitHubSignInWidget`

The `GitHubSignInWidget` handles the complete GitHub sign-in flow for your Flutter app.

You can customize the widget's appearance and behavior:

```

GitHubSignInWidget(
  client: client,
  // Button customization. The values shown are the defaults.
  style: GitHubButtonStyle.black, // or white
  size: SignInButtonSize.large, // or medium, small
  text: SignInButtonTextVariant.continueWith, // or signInWith, signUpWith, signIn
  shape: SignInButtonShape.pill, // or rounded, rectangular
  logoAlignment: SignInButtonLogoAlignment.center, // or left
  minimumWidth: 240, // at most 400
  textStyle: null, // TextStyle for the label

  // Scopes to request from GitHub
  // These are the default.
  scopes: const ['user', 'user:email', 'read:user'],

  onAuthenticated: () {
    // Do something when the user is authenticated.
    //
    // NOTE: You should not navigate to the home screen here, otherwise
    // the user will have to sign in again every time they open the app.
  },
  onError: (error) {
    // Handle errors
  },
)

```

NOTE

The `scopes` argument applies to **OAuth Apps**. For a **GitHub App**, the App's [Permissions](#) configured on the GitHub side control access and the `scopes` argument is ignored.

Build a custom UI with GitHubAuthController

For more control over the UI, you can use the `GitHubAuthController` class, which provides all the authentication logic without any UI components. This allows you to build a completely custom authentication interface.

```
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';

final controller = GitHubAuthController(
  client: client,
  onAuthenticated: () {
    // Do something when the user is authenticated.
    //
    // NOTE: You should not navigate to the home screen here, otherwise
    // the user will have to sign in again every time they open the app.
  },
  onError: (error) {
    // Handle errors
  },
  scopes: const ['user', 'user:email', 'read:user'],
);

// Initiate sign-in
await controller.signIn();
```

GitHubAuthController state management

Your widget should render the appropriate UI based on the `state` property of the controller. You can also use the below state properties to build your UI:

```
// Check current state
final state = controller.state; // GitHubAuthState enum

// Check if loading
final isLoading = controller.isLoading;

// Check if authenticated
final isAuthenticated = controller.isAuthenticated;

// Get error message
final errorMessage = controller.errorMessage;

// Listen to state changes
controller.addListener(() {
  setState(() {
    // Rebuild UI when controller state changes
  });
});
```

GitHubAuthController states

- `GitHubAuthState.idle` - Ready for user interaction.
- `GitHubAuthState.loading` - Processing a sign-in request.
- `GitHubAuthState.error` - An error occurred.
- `GitHubAuthState.authenticated` - Authentication was successful.

Related

- [Setup](#): configure GitHub sign-in on the server and in your app.
- [Troubleshooting](#): fix common GitHub sign-in errors.
- [UI components](#): style the sign-in buttons and localize the built-in UI.
- [Working with users](#): manage auth users and react to account events.